



University of Moratuwa

Department of Electronic & Telecommunication Engineering

EN2150 - Communication Network Engineering

Design of Routing Protocol

Adaptive Open Shortest Path First (AOSPF)

Team Synex

230033J	Amarasingha A.A.D.K.
230082G	Bandara W.D.A.C.
230164K	Dissanayake R.K.T.
230629R	Tennakoon U.G.R.B.

16th May 2026

Contents

1	Introduction	3
2	Baseline Protocols and Limitations	3
2.1	Routing Information Protocol (RIP)	3
2.2	Open Shortest Path First (OSPF)	4
2.3	Intermediate System to Intermediate System (IS-IS)	5
2.4	Border Gateway Protocol (BGP)	6
3	Proposed Protocol Design - Adaptive Open Shortest Path First (AOSPF)	7
3.1	Adaptive Composite Routing Metric	7
3.2	HELLO-Based Threshold Triggering Mechanism	8
3.3	Per-Link Advertised Cost Tracking	8
3.4	Modified Simulation Components	8
3.4.1	Original OSPF Cost Calculation	8
3.4.2	AOSPF Composite Cost Calculation	9
3.4.3	Original OSPF HELLO_ARRIVE Handler	9
3.4.4	AOSPF Enhanced HELLO_ARRIVE Handler	9
3.4.5	Per-Link Cost State Initialisation	10
3.5	Outside-Sense Convergence Discrepancy Detection	10
4	Route Computation with Adaptive Metrics	10
4.1	Modified SPF Computation	11
5	Failure Detection and Recovery	11
6	Security Enhancements	11
6.1	Overview of HMAC	12
6.2	Canonical Representation for AOSPF LSAs	12
6.3	HMAC Signing and Verification Procedures	12
6.4	How HMAC Improves Protocol Robustness	13
6.5	Security Impact on Adaptive Metrics	14
7	Simulation Implementation	14
8	Experiment Matrix and Parameters	16
8.1	Key Parameters	16
8.2	Performance Metrics	16
9	Results	16
9.1	Comparison: AOSPF vs OSPF	16
9.2	Specific Expectations	17
10	Limitations and Extensions	17
10.1	Limitations	17
10.2	Extensions	17
11	Router State Machine (Modified for AOSPF)	18

12 Conclusion	19
13 Individual Contribution	19
A Supplementary Materials	22

1. Introduction

This report presents a novel routing protocol that addresses the limitations of existing routing protocols by combining the strengths of them. The protocol merges OSPF's rapid link-state flooding with BGP's policy flexibility, while embedding security directly into the design. It introduces a multi-metric cost function that considers both real-time measured link delay and link bandwidth logarithmically, overcoming the single-metric constraints of traditional protocols. All control messages are authenticated and protected against replay attacks using HMAC-SHA256.

The project focuses on a simulated static wired network. Implementation uses Python with NetworkX and tkinter and assumes IPv4-like addressing. The simulator provides three operational modes: a baseline OSPF engine, an AOSPF engine without security, and a full AOSPF engine with HMAC-SHA256 protection. Deliverables include a design report, a functional simulator, and a security analysis. Design decisions are guided by RFCs and academic literature, with validation against RIP, OSPF, IS-IS, and BGP baselines.

2. Baseline Protocols and Limitations

We compare key legacy protocols to identify specific limitations to address.

2.1 Routing Information Protocol (RIP)

RIP is one of the oldest dynamic routing protocols still in use, primarily designed for small, homogeneous networks. Often described as the "simplest distance-vector protocol" or the "beginner's routing protocol," RIP allows routers to share reachability information based solely on hop count. Its main purpose is to provide a simple, loop-free path to destination networks within a limited network diameter.

Type:

RIP is a distance-vector protocol, which means each router maintains a table of destinations along with the distance (hop count) and direction (next-hop interface) to reach them. It operates over UDP port 520, sending periodic updates without establishing a persistent connection.

Scope:

RIP is strictly an IGP, designed for routing within a single Autonomous System (AS) or a small, standalone network. It is not used between major internet service providers or across the global internet backbone. RIPv1 is classful, broadcast-based and RIPv2 is classless, multicast-based with optional authentication. RIPng extends support to IPv6.

Metric:

RIP uses a single, simple metric: the number of routers (hops) between the source and destination network.

Best Path = Lowest Hop Count Maximum Hops = 15 (Any route requiring 16 or more hops is considered unreachable. This hard limit caps the maximum network diameter at 15 routers.)

Key Features:

- Extreme Simplicity: Configuration typically requires only enabling RIP on interfaces

and advertising networks.

- Low Resource Footprint: Requires minimal CPU and memory.
- Widely Supported
- Built-in Loop Prevention: Uses split horizon, route poisoning, and hold-down timers
- VLSM & CIDR Support (RIPv2+)

Limitations:

- Poor Scalability: The 15-hop limit makes RIP unusable for all but the smallest networks
- Slow Convergence: Periodic full-table updates every 30 seconds combined with hold-down timers (180 seconds) mean network failures can take minutes to resolve, causing temporary black-holes or loops.
- Inefficient Metric: Hop count ignores critical factors like link speed, congestion, packet loss, and administrative cost.
- Chatty Behavior: Broadcasting or multicasting the entire routing table every 30 seconds wastes bandwidth on modern high-speed networks.
- No Authentication in RIPv1
- No Support for Hierarchical Design

2.2 Open Shortest Path First (OSPF)

Type:

OSPF is a Link-State routing protocol. It is an Interior Gateway Protocol (IGP), meaning it is designed to move data within a single internal network or Autonomous System (AS). It uses the Shortest Path First (SPF) or Dijkstra's algorithm to calculate the loop-free shortest path to every node in the network.

Scope:

OSPF uses a two-layer hierarchy consisting of Areas. The core of the OSPF network is Area 0 (Backbone). All other areas must physically or logically connect to Area 0 to exchange routing information. By dividing the network into areas, OSPF limits the size of the Link-State Database (LSDB) on individual routers, reducing the processing load and containing the impact of network "flapping" (constant status changes) to a specific area.

Metric:

OSPF uses Cost as its metric. Cost is an arbitrary value inversely proportional to the bandwidth of an interface. The faster the link, the lower the cost.

$$\text{Cost} = \text{Reference Bandwidth} / \text{Interface Bandwidth}$$

Default Reference: Typically 100 Mbps. This means a 10 Mbps link has a cost of 10, while a 100 Mbps link has a cost of 1.

Key Features:

- Fast Convergence: Because routers share link-state information rather than just routing tables, they can recalculate paths very quickly when a change is detected.
- Classless Support: Fully supports VLSM and CIDR.
- Multicast Updates: Uses multicast addresses (224.0.0.5 for all SPF routers and 224.0.0.6 for Designated Routers) to send updates.

- Event-Driven Updates: only sends updates when a change occurs in the topology (LSU).
- Authentication: Supports cleartext, MD5, and SHA authentication for secure routing updates between neighbors.

Limitations:

- Running the SPF algorithm requires significant CPU power and memory specially in large and flat (single-area) networks with many routers.
- Setting up multi-area OSPF and troubleshooting virtual links or summarization can be more difficult than simpler protocols like RIP.
- Strict Design Requirements: The requirement that all areas must connect to Area 0 can make network expansion rigid if the physical topology doesn't naturally support a "hub-and-spoke" area design.
- Reference Bandwidth Issues: Since the default reference bandwidth is 100 Mbps OSPF treats 100 Mbps, 1 Gbps, and 10 Gbps links all as a Cost of 1 unless the reference bandwidth is manually adjusted.

2.3 Intermediate System to Intermediate System (IS-IS)

IS-IS is a high-performance, link-state routing protocol. Originally created by the ISO for the OSI protocol suite, it was later adapted to support IP, becoming the backbone of choice for many of the world's largest Service Providers. It is prized for its extreme scalability, protocol-agnostic nature, and high stability under heavy load.

Type:

It uses Dijkstra's SPF (Shortest Path First) algorithm to calculate the best path. Each router maintains a Link State Database (LSDB) to ensure a synchronized view of the network. Unlike OSPF, it operates natively at Layer 2, sending updates directly over the Data Link layer without requiring an IP header.

Scope:

IS-IS is an Interior Gateway Protocol (IGP) used for routing within a single Autonomous System. While it can scale to massive sizes, it is predominantly found in Service Provider cores and large-scale Data Centers. It uses a flexible level-based hierarchy (Level 1 for intra-area and Level 2 for inter-area backbone) rather than a rigid area-centric design.

Metric:

IS-IS uses a dimensionless Interface Cost metric.

Default Metric: Almost all implementations default to a cost of 10 per interface, regardless of bandwidth. Wide Metrics: Modern networks must use "Wide Metrics" (24-bit/32-bit values) to support high-speed links and advanced traffic engineering.

Key Features:

- Manages traffic by dividing the network into logical Areas. Unlike other protocols where individual interfaces are assigned to an area, IS-IS assigns the entire router to a specific area.
- Protocol Agnostic: Because it runs at Layer 2, IS-IS can carry routing information for IPv4, IPv6, and non-IP protocols simultaneously using the same adjacency.
- Partial Route Calculation (PRC): It can update specific prefix changes (leaf nodes)

without performing a full, CPU-heavy recalculation of the entire network map.

- **TLV Extensibility:** It uses Type-Length-Value data fields, allowing engineers to add new features without needing to overhaul the core protocol
- **High Stability:** Its independence from the IP stack makes it resistant to IP-based spoofing or DoS attacks and ensures it stays up even if IP configurations fail.

Limitations:

- **Operational Complexity:** Requires the configuration of NET addresses, which follow a non-IP ISO format that is often unfamiliar to enterprise engineers.
- **Troubleshooting Hurdles:** Standard IP-based tools (like ping/traceroute) cannot be used to verify the protocol's own adjacency; specialized Layer 2 commands are required.
- **Suboptimal Defaults:** The default narrow metrics and fixed cost of 10 can lead to poor routing decisions on high-speed links if not manually tuned to wide metrics.
- **Learning Curve:** It is generally considered more difficult to master than OSPF, leading to a smaller pool of qualified engineers for support.

2.4 Border Gateway Protocol (BGP)

BGP is the primary routing protocol of the global internet, often described as the “postal service” or the “glue” that holds the digital universe together. It manages how data packets are delivered between massive networks known as Autonomous Systems (ASs).

Type:

BGP is classified as a path-vector protocol. It operates over reliable TCP sessions (typically on port 179).

Scope:

Its primary role is as an Exterior Gateway Protocol (EGP) for inter-AS routing, though it is also used internally (iBGP) to ensure all routers within an AS share consistent information.

Metric:

BGP does not use traditional metrics like bandwidth or delay; instead, it uses a complex hierarchy of Path Attributes (PAs) and rule-sets to select routes. The AS_PATH is a primary attribute used for path selection and loop prevention.

Features:

- It is the only protocol capable of managing the massive global internet routing table, which now exceeds hundreds of thousands of prefixes.
- It offers granular policy-based routing control and supports multi-homing for high availability and redundancy.

Limitations:

- BGP has very slow convergence times because routers explore many invalid paths before stabilizing after a change.
- It lacks intrinsic security, making it highly vulnerable to prefix hijacking, spoofing, and TCP reset attacks.
- Routing decisions are often based on coarse attributes like the number of AS hops rather than actual link performance metrics.

3. Proposed Protocol Design - Adaptive Open Shortest Path First (AOSPF)

AOSPF extends the existing OSPF simulation with three major enhancements: an adaptive composite cost metric derived from real-time HELLO round-trip delay measurements and a logarithmic bandwidth term, a HELLO-based threshold triggering mechanism that floods LS-UPDATE packets when the computed cost deviates significantly from the last advertised value, and an HMAC-SHA256 security layer that authenticates all LSAs.

3.1 Adaptive Composite Routing Metric

Define the composite cost for a link as:

$$\text{Cost} = \left\lceil w_1 \cdot \frac{L_{\text{measured}}}{L_{\text{max}}} + w_2 \cdot \left(-\log \left(\frac{BW}{BW_{\text{max}}} \right) \right) \right\rceil$$

where:

- L_{measured} = one-way delay measured from HELLO packet transit time (ms)
- BW = link bandwidth (Mbps)
- L_{max} = normalization constant (maximum expected delay)
- BW_{max} = normalization constant (maximum reference bandwidth)
- w_1, w_2 = configurable weights

The logarithmic bandwidth term $-\log(BW/BW_{\text{max}})$ is always non-negative since $BW \leq BW_{\text{max}}$, and grows as bandwidth decreases, correctly penalising lower-capacity links. The delay L_{measured} is obtained passively: each HELLO packet records its transmission timestamp, and the receiving router computes the one-way transit time as the difference between arrival time and the embedded sent time. This requires no dedicated probe traffic beyond the standard keepalive mechanism.

Comparison with OSPF Cost:

Aspect	Traditional OSPF	AOSPF
Metric Formula	$\lceil \text{REF_BW}/BW \rceil$	$\lceil w_1(L/L_{\text{max}}) + w_2(-\log(BW/BW_{\text{max}})) \rceil$
Delay Awareness	None	Measured per HELLO transit
Bandwidth Handling	Linear inverse	Logarithmic (better high-speed differentiation)
Adaptivity	Static (manual reconfiguration)	Automatic threshold-based updates
Flooding Triggers	Topology changes only	Topology changes + cost deviation $\geq 40\%$
Multi-criteria	Single metric	Weighted delay + logarithmic bandwidth

3.2 HELLO-Based Threshold Triggering Mechanism

The core innovation of AOSPF is that LS-UPDATE packets are flooded when the newly computed composite cost deviates by 40% or more from the last advertised cost on that link. This check is performed at every HELLO arrival, piggybacking on the existing keepalive mechanism at no additional control-plane overhead.

Threshold Logic:

Algorithm 1: AOSPF HELLO-Triggered Cost Evaluation

Input: sender, receiver, sent_time, current_time, w_1 , w_2 , $L_{\max}$, $BW_{\max}$

```

1  $L_{\text{measured}} \leftarrow \text{current\_time} - \text{sent\_time};$ 
2  $BW \leftarrow \text{get\_link\_bandwidth}(\text{receiver}, \text{sender});$ 
3  $\text{bw\_ratio} \leftarrow \max(BW/BW_{\max}, \epsilon);$  // guard against log(0)
4  $C_{\text{new}} \leftarrow \max\left(1, \left\lceil w_1 \cdot \frac{L_{\text{measured}}}{L_{\max}} + w_2 \cdot (-\ln(\text{bw\_ratio})) \right\rceil\right);$ 
5  $C_{\text{old}} \leftarrow \text{advertised\_costs}[\text{receiver}][\text{sender}];$ 
6 if adjacency is 2-WAY then
7   if  $C_{\text{new}} \geq 1.4 \times C_{\text{old}}$  or  $C_{\text{new}} \leq 0.6 \times C_{\text{old}}$  then
8      $\text{advertised\_costs}[\text{receiver}][\text{sender}] \leftarrow C_{\text{new}};$ 
9     Increment LSA sequence number for receiver;
10    Build new LSA with updated neighbor cost map;
11    Sign LSA with HMAC (secured mode);
12    Flood LSA to all 2-WAY neighbors;
```

Both an increase of $\geq 40\%$ (congestion worsening) and a decrease of $\geq 40\%$ (congestion clearing) trigger an update, ensuring the LSDB tracks cost improvements as well as degradations. Triggering is naturally rate-limited by the HELLO interval (configurable: 1000, 2000, 3000, or 5000 ms); cost can only be re-evaluated as frequently as HELLOs arrive.

3.3 Per-Link Advertised Cost Tracking

Each router maintains a local table $\text{advertised_costs}[\text{node}][\text{neighbor}]$ that stores the composite cost last flooded for each incident link. This table is initialised at simulation start from the static OSPF base costs and is updated each time a threshold crossing causes a new LSA to be flooded. All LSAs carry these per-link composite costs in their neighbor maps, so Dijkstra runs on dynamically updated weights rather than fixed values.

3.4 Modified Simulation Components

The AOSPF simulation extends the baseline OSPF discrete-event simulator with the following structural changes.

3.4.1 Original OSPF Cost Calculation

In the baseline OSPF engine, link cost is computed once at topology load time from the static reference bandwidth and is never updated during simulation:

Algorithm 2: OSPF Static Cost Initialisation

Input: link bandwidth BW (Mbps), reference bandwidth $\text{REF} = 1000$ Mbps

```

1  $\text{cost} \leftarrow \max(1, \lceil \text{REF}/BW \rceil);$ 
2 Store cost in graph edge and original_edges_data;
```

3.4.2 AOSPF Composite Cost Calculation

AOSPF replaces the static cost with a dynamic formula evaluated on every HELLO arrival. The static edge cost stored at load time is no longer used for routing; the composite cost computed from live measurements governs all LSA advertisements.

Algorithm 3: AOSPF Composite Cost (evaluated per HELLO arrival)

Input: L_{measured} (ms), BW (Mbps), weights w_1, w_2 , $L_{\text{max}} = 50$ ms,
 $BW_{\text{max}} = 1000$ Mbps

```

1 bw_ratio  $\leftarrow \max(BW/BW_{\text{max}}, \epsilon)$ ;
2 cost  $\leftarrow w_1 \cdot (L_{\text{measured}}/L_{\text{max}}) + w_2 \cdot (-\ln(\text{bw\_ratio}))$ ;
3 return  $\max(1, \lceil \text{cost} \rceil)$ ;

```

3.4.3 Original OSPF HELLO_ARRIVE Handler

In the baseline OSPF engine, HELLO packets are used solely for adjacency establishment and dead-timer refreshing. No metric information is extracted from transit timing, and no LSA is flooded unless a new 2-WAY adjacency forms.

3.4.4 AOSPF Enhanced HELLO_ARRIVE Handler

AOSPF extends the HELLO arrival handler with cost evaluation and conditional LSA flooding:

Algorithm 4: AOSPF HELLO_ARRIVE Event Handler

Input: sender, receiver, sender_neighbor_list, link_delay, sent_time

```

1 Refresh dead timer for sender at receiver;
2 Compute  $C_{\text{new}}$  using Algorithm 2;
3  $C_{\text{old}} \leftarrow \text{advertised\_costs}[\text{receiver}][\text{sender}]$ ;
4 Log cost evaluation details to local router event trace;
5 if  $\text{receiver} \in \text{sender\_neighbor\_list}$  then
6   if  $\text{adjacency} \neq \text{2-WAY}$  then
7     Set adjacency to 2-WAY;
8      $\text{advertised\_costs}[\text{receiver}][\text{sender}] \leftarrow C_{\text{new}}$ ;
9     Generate and sign new LSA with updated neighbor cost map;
10    Flood LSA to all 2-WAY neighbors;
11    Exchange all cached LSAs with new neighbor (database exchange);
12  else
13    // Existing 2-WAY adjacency - check threshold
14    if  $C_{\text{new}} \geq 1.4 \times C_{\text{old}}$  or  $C_{\text{new}} \leq 0.6 \times C_{\text{old}}$  then
15       $\text{advertised\_costs}[\text{receiver}][\text{sender}] \leftarrow C_{\text{new}}$ ;
16      Generate and sign updated LSA;
17      Flood LSA to all 2-WAY neighbors;
18      Record threshold-crossing event for convergence metrics log;
19 else
20   if  $\text{adjacency} = \text{DOWN}$  then
21     Transition adjacency to INIT;
22     Schedule reactive HELLO response;

```

3.4.5 Per-Link Cost State Initialisation

Algorithm 5: Advertised Cost Table Initialisation

```

1 for each node  $n$  in topology do
2   for each neighbor  $k$  of  $n$  do
3      $\text{advertised\_costs}[n][k] \leftarrow \text{static\_ospf\_cost}(n, k);$ 
// Table is updated in-place whenever a threshold crossing triggers a
  new LSA

```

3.5 Outside-Sense Convergence Discrepancy Detection

The simulation includes an omniscient observer that evaluates at every millisecond whether the currently advertised costs remain accurate relative to the ground-truth delay. For every active link, it computes the true composite cost from the current delay and checks whether any advertised cost deviates beyond the 40% threshold:

Algorithm 6: Outside-Sense Delay-Cost Discrepancy Check

```

1 discrepancy  $\leftarrow$  false;
2 for each active link  $(u, v)$  do
3    $L \leftarrow \text{get\_current\_delay}(u, v, t);$ 
4    $BW \leftarrow \text{get\_link\_bandwidth}(u, v);$ 
5    $C_{\text{true}} \leftarrow \text{ComputeCompositeCost}(L, BW);$  // Algorithm 2
6   for each direction  $(a, b) \in \{(u, v), (v, u)\}$  do
7      $C_{\text{adv}} \leftarrow \text{advertised\_costs}[a][b];$ 
8     if  $C_{\text{true}} \geq 1.4 \times C_{\text{adv}}$  or  $C_{\text{true}} \leq 0.6 \times C_{\text{adv}}$  then
9       discrepancy  $\leftarrow$  true;
10 return discrepancy;

```

When this flag is set, the convergence indicator transitions to a “Stable but Inaccurate (Delay Cost Discrepancy)” yellow state, distinct from a topology-level inaccuracy, and the network is not considered truly converged until the next HELLO cycle detects and propagates the update.

4. Route Computation with Adaptive Metrics

Each router maintains a Link-State Database (LSDB) containing topology information. When an LS-UPDATE is received, whether triggered by a topology change (adjacency up/down) or a threshold-crossing cost update, the router installs the new LSA and Dijkstra’s algorithm is re-run using the composite costs stored in each LSA’s neighbor map.

4.1 Modified SPF Computation

Algorithm 7: AOSPF Shortest Path First Computation

Input: source router s , LSDB at router s

- 1 Build local graph G' from LSDB;
 - 2 **for** each LSA entry $(u, \text{neighbors})$ in LSDB **do**
 - 3 **for** each (v, cost) in neighbors **do**
 - 4 Add edge (u, v) with weight = cost to G' ;
 - 5 Run Dijkstra's algorithm on G' from source s ;
 - 6 **return** shortest-path distances and next-hop routing table;
-

The LSA neighbor map stores the advertised composite cost at the time of flooding; therefore Dijkstra automatically operates on delay-aware, bandwidth-sensitive costs without any additional post-processing.

5. Failure Detection and Recovery

AOSPF inherits OSPF's failure detection mechanisms and enhances them:

- **HELLO/Keepalive Timers:** Periodic HELLO packets every `hello_interval` (configurable: 1000, 2000, 3000, or 5000 ms). Missed HELLOs within `dead_interval` ($4 \times \text{hello_interval}$) trigger link-down event and adjacency teardown.
- **HELLO-Based Metric Reactivity:** On every HELLO arrival the router recomputes the composite cost from the measured transit delay and link bandwidth. If the deviation from the last advertised cost exceeds 40%, the router immediately floods an updated LSA, allowing routes to adapt to congestion before an outright failure occurs.
- **Triggered LS Updates on Topology Change:** On link-down (dead timer expiry), the affected router generates a new LSA omitting the failed neighbor and floods it to all remaining 2-WAY adjacencies.
- **Triggered LS Updates on Cost Change:** When the 40% threshold is crossed during a HELLO exchange, the detecting router similarly increments its LSA sequence number and refloods without waiting for the next scheduled topology event.
- **Convergence Metrics Logging:** The simulation records the time from physical link failure to dead-timer expiry, and from dead-timer expiry to full LSDB re-synchronisation. For cost-metric changes it records the time from delay alteration to threshold detection, and from detection to LSDB convergence.

6. Security Enhancements

AOSPF incorporates a comprehensive security layer based on Hash-based Message Authentication Code (HMAC) to protect against routing attacks. This section details the HMAC mechanism and explains how it improves protocol robustness. The implementation ships as two simulator variants: an insecure mode (no HMAC, all LSAs accepted) used to demonstrate attack impact, and a secured mode (HMAC-SHA256) used to demonstrate

attack prevention.

6.1 Overview of HMAC

HMAC (Hash-based Message Authentication Code) is a keyed cryptographic hash function that provides both message authentication and integrity verification. In AOSPF, all LSAs (generated during adjacency formation, cost threshold crossings, and topology changes) are signed using HMAC-SHA256 with a pre-shared secret key known only to legitimate routers. The HMAC computation is defined as:

$$\text{HMAC}(K, m) = \text{SHA256}\left((K \oplus \text{opad}) \parallel \text{SHA256}((K \oplus \text{ipad}) \parallel m)\right)$$

where:

- K is the shared secret key (32-byte random key generated at process start)
- m is the canonical representation of the LSA fields
- `ipad` and `opad` are fixed padding constants
- `||` denotes concatenation

6.2 Canonical Representation for AOSPF LSAs

To ensure deterministic HMAC computation, AOSPF constructs a canonical string from the LSA's immutable fields before signing. The canonical form concatenates the router ID, sequence number, TTL, and the sorted list of neighbor-cost pairs:

$$m = \text{router_id} \parallel \text{sequence_num} \parallel \text{ttl} \parallel \text{sorted}(\text{neighbors.items}())$$

The sorted order of neighbor items guarantees that two semantically identical LSAs produce the same HMAC tag regardless of dictionary insertion order. The HMAC tag is appended to the LSA as a separate field before transmission.

6.3 HMAC Signing and Verification Procedures

Algorithm 8: SignLSA - generate HMAC tag before transmission

Input: LSA payload P , shared key K

Output: Signed LSA P' with HMAC tag appended

- 1 $m \leftarrow \text{BuildCanonical}(P)$;
 - 2 $\text{tag} \leftarrow \text{HMAC-SHA256}(K, m)$;
 - 3 $P' \leftarrow P \cup \{\text{hmac} : \text{tag}\}$;
 - 4 **return** P' ;
-

Algorithm 9: VerifyLSA - authenticate and integrity-check an incoming LSA

Input: Received LSA P_{rcv} , shared key K **Output:** Boolean (valid/invalid) and reason string

```

1 if  $hmac \notin P_{\text{rcv}}$  then
2   return (False, "REJECTED - No HMAC tag present (unsigned / forged packet)");
3  $m \leftarrow \text{BuildCanonical}(P_{\text{rcv}})$ ;
4  $\text{expected} \leftarrow \text{HMAC-SHA256}(K, m)$ ;
5 if  $\text{ConstantTimeCompare}(\text{expected}, P_{\text{rcv}}.hmac)$  then
6   return (True, "ACCEPTED - HMAC-SHA256 verified");
7 else
8   return (False, "REJECTED - HMAC mismatch (forged or tampered)");

```

6.4 How HMAC Improves Protocol Robustness

Compared to an unprotected OSPF implementation (as in the insecure simulation variant), the integration of HMAC provides the following security guarantees:

1. **Authentication of Origin:** A router can cryptographically verify that an LSA originated from a legitimate router that possesses the shared secret. Without HMAC, any attacker on a broadcast link can inject forged LSAs claiming to be any router.
2. **Integrity Protection:** The HMAC covers the router ID, sequence number, TTL, and the full sorted neighbor-cost map. Any modification of these fields causes verification to fail, preventing man-in-the-middle attacks that alter routing information in transit.
3. **Replay Attack Prevention:** The LSA header includes a monotonically increasing sequence number. Combined with HMAC, receivers can detect and discard replayed packets because the sequence number is covered by the tag and cannot be forged without the key.
4. **Early Drop of Forged Packets:** Invalid LSAs are discarded immediately at the LSA arrival handler before any LSDB update or re-flooding occurs, so malicious packets consume minimal processing resources and cannot propagate through the network.
5. **Protection Against Metric Manipulation:** In the insecure simulation, an attacker injects a fake LSA with artificially low costs to hijack routing, and the compromised routing tables are highlighted in the UI. With HMAC, such a packet is rejected at every router because it carries no valid tag. This directly addresses the vulnerability demonstrated in the insecure variant.
6. **Forensic Traceability:** Every rejected packet is logged with the reason (no HMAC tag, HMAC mismatch) and highlighted in the Global Commentary Log with a distinct visual indicator. Accepted legitimate LSAs are similarly tagged, providing network operators with clear evidence of attempted attacks.

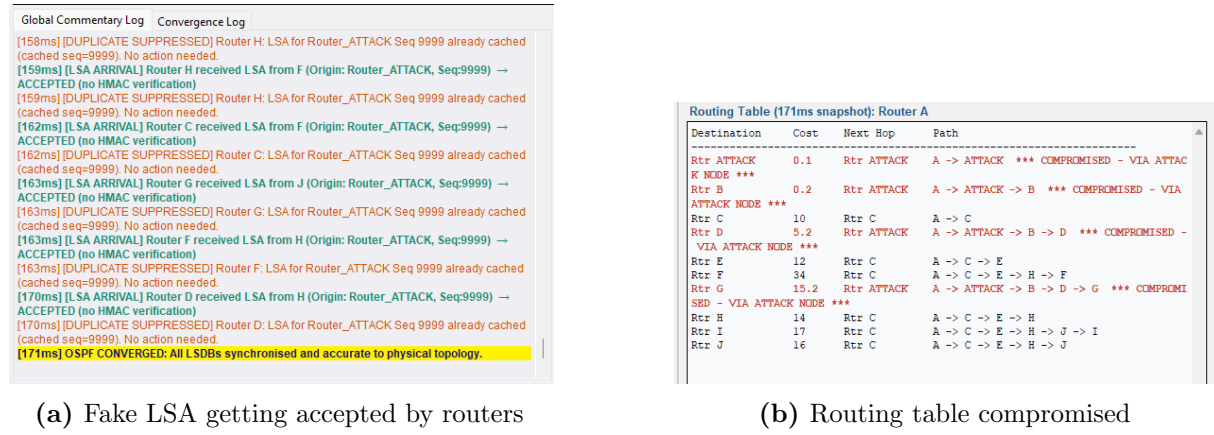


Figure 1: Without HMAC verification

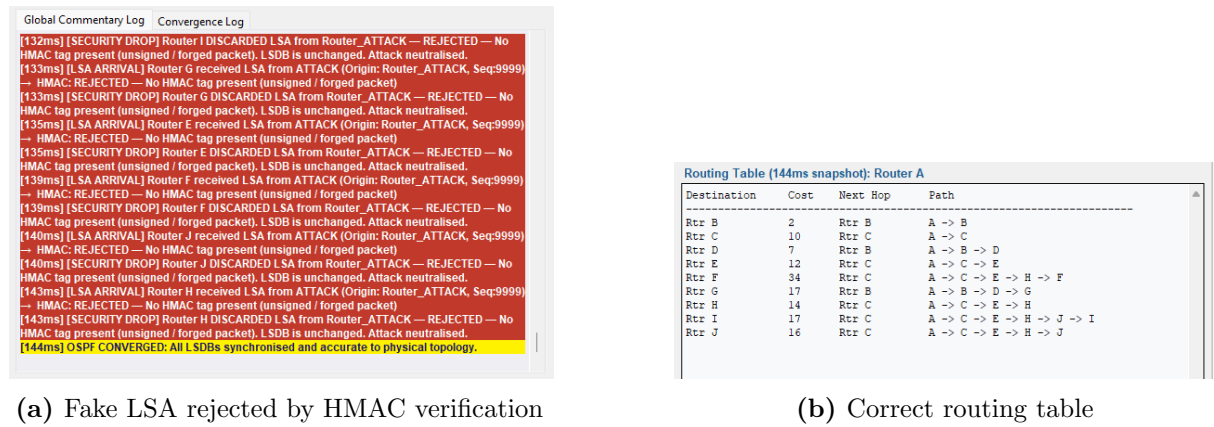


Figure 2: With HMAC verification

6.5 Security Impact on Adaptive Metrics

All LSAs generated by the adaptive metric system - whether triggered by adjacency formation, cost-threshold crossings, or dead-timer expiry - are signed before transmission. This ensures that even the dynamic, autonomously triggered metric updates cannot be spoofed by an attacker. An adversary cannot force a routing change by injecting a fabricated threshold-crossing LSA because the forged packet would lack a valid HMAC tag and be dropped at every receiving router.

7. Simulation Implementation

The complete simulation implementation, including all source code, configuration files, and experiment scripts, is available at:

<https://github.com/Achinta-Bandara/Routing-Protocol-Design.git>

The simulator is structured as four standalone Python scripts sharing a common topology files:

- **Baseline OSPF engine** - standard link-state flooding with static bandwidth-derived costs.

- **OSPF engine (insecure)** - standard OSPF to demonstrate security vulnerability; includes a “Inject Fake LSA” button to demonstrate the attack surface.
- **Basic AOSPF engine** - composite log-bandwidth cost metric and HELLO-based threshold triggering, no HMAC.
- **AOSPF engine (secured)** - full AOSPF with HMAC-SHA256 signing on every LSA; the same attack button demonstrates that forged LSAs are blocked.

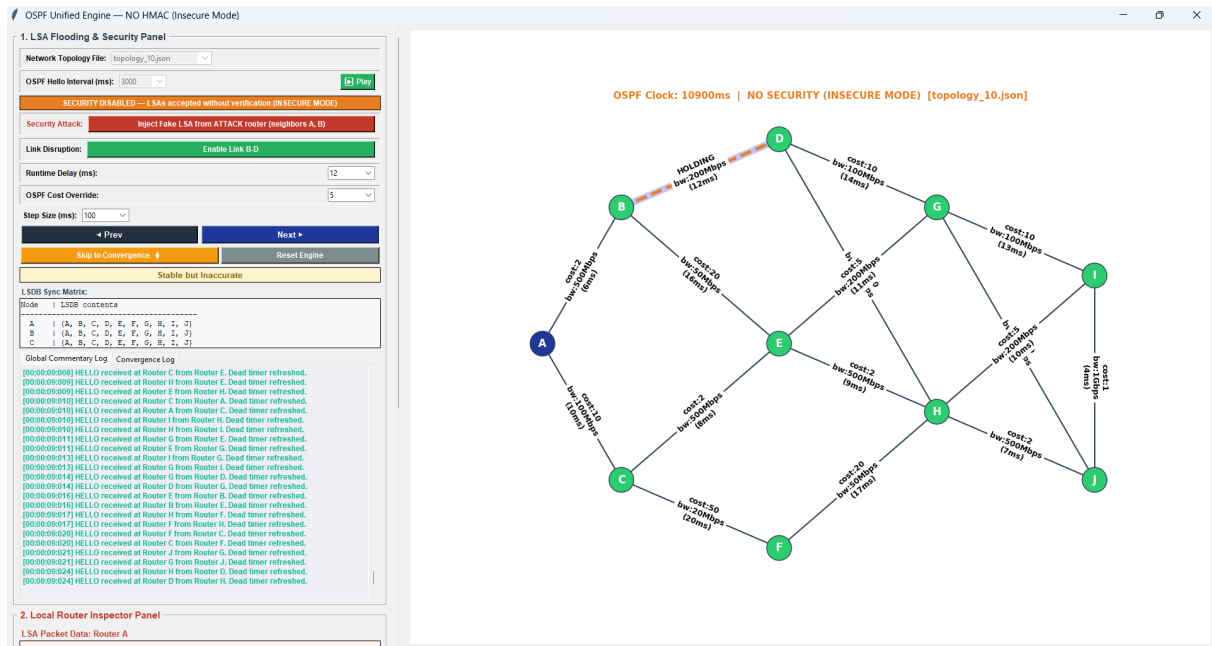


Figure 3: OSPF simulator

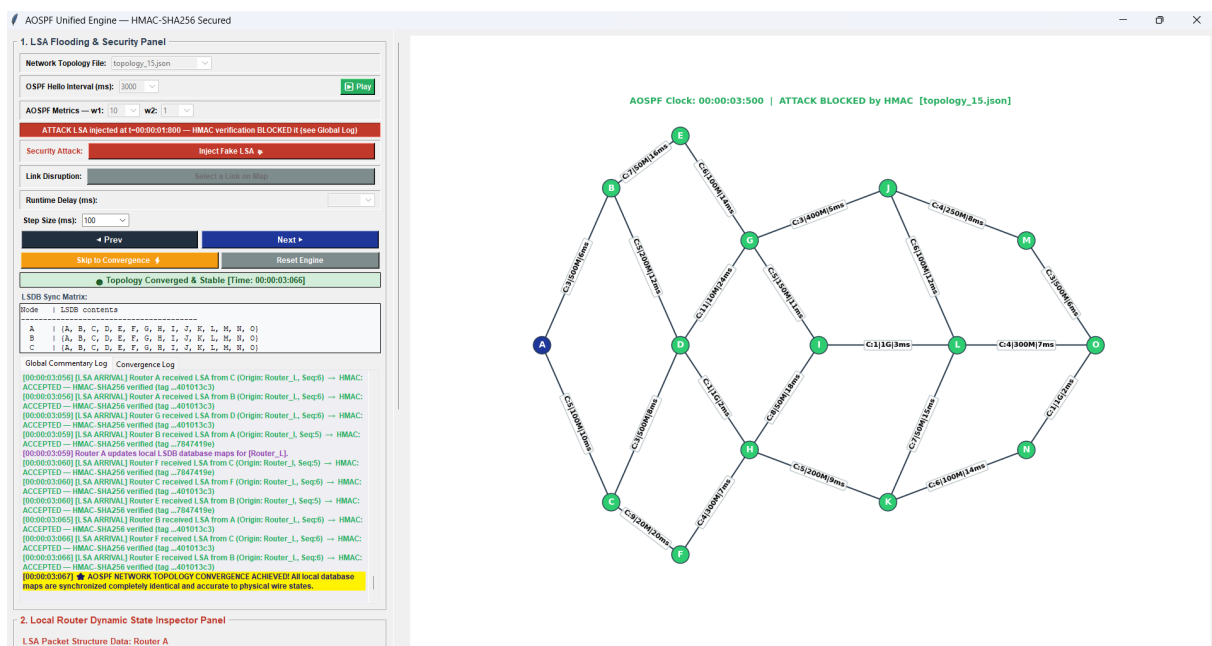


Figure 4: AOSPF simulator

8. Experiment Matrix and Parameters

8.1 Key Parameters

1. Network Size (N): 5, 10, 15 nodes
2. Topology Density: sparse, moderate, dense
3. Metric Weights: $(w_1, w_2) = (0,1)$ bandwidth-only, $(1,1)$ balanced, $(10,1)$ delay-dominant, $(0,0)$ baseline (zero cost)
4. Link Failure Rate: 0%, 5%, 10%
5. Hello Interval: 1000 ms, 2000 ms, 3000 ms, 5000 ms

8.2 Performance Metrics

- **Convergence Time:** Time from topology change or delay modification until all routers have a consistent, physically accurate LSDB.
- **Metric Detection Latency:** Time from a runtime delay change to threshold detection by the adjacent router (bounded by the HELLO interval).
- **Post-Detection Convergence Time:** Time from threshold detection to full LSDB re-synchronisation across all nodes.
- **Control Overhead:** Number of LS-UPDATE packets generated, separated by trigger type (topology change vs. cost-threshold crossing).
- **Route Optimality:** Average composite cost of selected routes compared to the globally optimal path computed with ground-truth delays and bandwidths.
- **Security Effectiveness:** Number of forged LSAs injected vs. blocked, and presence or absence of the ATTACK router in routing tables.

9. Results

9.1 Comparison: AOSPF vs OSPF

Metric	OSPF	AOSPF
Convergence after failure	Fast (link-state flooding)	Fast (link-state flooding)
Adaptation to congestion	None (static costs)	Next HELLO after delay change
Path quality with congestion	Poor (uses static costs)	Good (adapts to measured delay and bandwidth)
Control overhead (steady)	Low (event-driven)	Low (no extra events; reuses HELLOs)
Control overhead (congestion)	Low (no updates)	Moderate (one LSA flood per threshold crossing)
High-speed link differentiation	Poor (linear inverse)	Good (logarithmic bandwidth term)

9.2 Specific Expectations

1. **Faster Adaptation to Congestion:** When link delay increases due to congestion, AOSPF detects the change at the next HELLO arrival and floods an LS-UPDATE if the 40% threshold is crossed. OSPF never adapts to delay or bandwidth changes without manual reconfiguration.
2. **Better High-Speed Link Differentiation:** The logarithmic bandwidth term $-\log(BW/BW_{\max})$ produces meaningful cost differences between high-speed links (e.g., 500 Mbps vs. 1000 Mbps) that the OSPF linear-inverse formula collapses to the same cost.
3. **Controlled Overhead:** With the 40% threshold, links whose measured delay fluctuates by less than 40% of the last advertised value generate no additional LSA floods. Only sustained or large delay or bandwidth shifts produce updates.
4. **Tunable Path Selection:** By adjusting w_1 and w_2 , operators can weight routing toward low latency ($w_1 \gg w_2$), toward high bandwidth ($w_2 \gg w_1$), or any combination. Setting $w_1 = 0$ recovers a pure bandwidth-driven policy.
5. **Convergence Time:** For topology changes (link failure/restoration), convergence behaviour is identical to OSPF. For metric-triggered changes, total convergence equals HELLO interval (detection latency) plus LSA flooding time plus SPF computation time.

10. Limitations and Extensions

10.1 Limitations

- Delay measurement relies on HELLO transit time, which conflates propagation delay with queuing and processing delays; the measurement is not isolated to propagation alone.
- The 40% threshold is a fixed constant in the simulation; optimal values may differ per link type or traffic pattern.
- Metric detection latency is bounded below by the HELLO interval; sub-interval congestion events are not detected until the next HELLO.
- Weights w_1 and w_2 are set once at simulation start and remain static for the entire run.
- Security uses a single pre-shared key distributed out-of-band; no PKI or per-adjacency key management is implemented.
- The simulation models only static wired topologies; mobile or wireless extensions are not included in the current implementation.

10.2 Extensions

- Adaptive threshold tuning based on observed delay variance per link.
- Per-adjacency key rotation and PKI-based key distribution to replace the static pre-shared key.
- Sub-HELLO delay probing using lightweight probe packets to reduce detection

latency without increasing HELLO frequency.

- Dynamic weight adjustment based on real-time network conditions or operator policy.
- Multi-metric extension: incorporate packet loss rate, jitter, and residual bandwidth into the composite cost.
- Integration with SDN controllers for centralised threshold and weight optimisation.
- Implementation in real router platforms (FRRouting, Quagga) for hardware validation.

11. Router State Machine (Modified for AOSPF)

Algorithm 10: AOSPF Router State Machine (key transitions)

```

1 on HELLO timer expiry do
2   | Send HELLO with current neighbour list and timestamp;
3 on HELLO_ARRIVE from sender do
4   | Compute composite cost  $C_{\text{new}}$  via log-bandwidth formula;
5   | if new 2-WAY adjacency then
6   |   | Transition to 2-WAY; flood new LSA; run database exchange;
7   | else if  $|C_{\text{new}} - C_{\text{old}}|/C_{\text{old}} \geq 40\%$  then
8   |   | Update advertised cost; flood updated LSA (cost-triggered);
9   | else
10  |   | Refresh dead timer only;
11 on LSA_ARRIVE do
12  | Verify HMAC (secured mode); drop if invalid;
13  | if sequence number > cached then
14  |   | Install in LSDB; rerun SPF; re-flood to other 2-WAY neighbours;
15 on dead timer expiry for neighbour v do
16  | Tear down adjacency; generate corrective LSA removing  $v$ ; flood;

```

New states and transitions compared to OSPF:

- **Cost Evaluation on HELLO:** Composite log-bandwidth cost is computed on every HELLO arrival and compared against the last advertised value; a $\geq 40\%$ deviation triggers an immediate LSA flood.
- **HMAC Gate on LSA_ARRIVE:** In the secured variant, every arriving LSA passes through HMAC verification before any LSDB update or re-flooding; invalid packets are silently discarded.
- **Cost-Triggered LSA Flood:** A standard LSA with an incremented sequence number carrying the updated composite cost is flooded; no new packet type or header flag is required.

12. Conclusion

This report presented Adaptive Open Shortest Path First (AOSPF), an extension to traditional OSPF that introduces:

1. A composite cost function combining real-time measured HELLO transit delay and a logarithmic bandwidth term, weighted by configurable parameters w_1 and w_2 : $\lceil w_1 \cdot L/L_{\max} + w_2 \cdot (-\log(BW/BW_{\max})) \rceil$.
2. A HELLO-based threshold triggering mechanism: when the composite cost on a link deviates by $\geq 40\%$ from the last advertised value, the detecting router immediately floods an updated LSA, enabling dynamic route optimisation without dedicated probe traffic.
3. Per-link advertised cost tracking that allows Dijkstra's SPF to run on dynamically updated, delay- and bandwidth-aware weights at every convergence event.
4. An HMAC-SHA256 security layer that signs every LSA on transmission and rejects any unsigned or tampered packet at the receiving router, neutralising LSA injection attacks demonstrated in the insecure simulation variant.

The implementation extends an existing OSPF discrete-event simulation written in Python with NetworkX and tkinter. Key changes include embedding the log-bandwidth cost formula and threshold evaluation inside the `HELLO_ARRIVE` handler, initialising and maintaining per-link `advertised_costs`, extending the convergence observer with an outside-sense delay-cost discrepancy check, and adding `sign_lsa` and `verify_lsa` functions in the secured variant.

AOSPF addresses OSPF's fundamental limitation: the inability to react to changing link conditions without manual intervention. By piggybacking cost evaluation onto the existing HELLO mechanism, AOSPF achieves delay- and bandwidth-aware adaptive routing with zero additional control-plane message types, while the HMAC layer ensures that the dynamic update mechanism cannot be exploited by adversarial LSA injection.

13. Individual Contribution

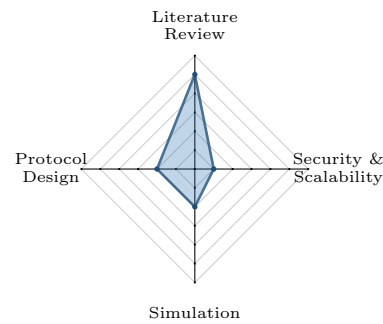
A.A.D.K. Amarasinghe - 230033J

Main Area: Literature Review

- **Literature Review:** Conducted detailed analysis of RIP, OSPF, IS-IS and BGP routing protocols. Identified operational limitations, convergence issues, and routing inefficiencies. Prepared comparative technical summaries and documentation. Contributed significantly to the theoretical foundation.
- **Protocol Design:** Assisted in designing protocol workflows and control message structures. Helped develop state management logic and flow diagrams.
- **Simulation:** Assisted in reviewing simulation results and validating correctness. Contributed to interpreting performance metrics and outputs.
- **Security & Scalability:** Assisted in identifying protocol weaknesses and reliability issues. Contributed suggestions for improving routing stability and scalability.

Contribution Breakdown:

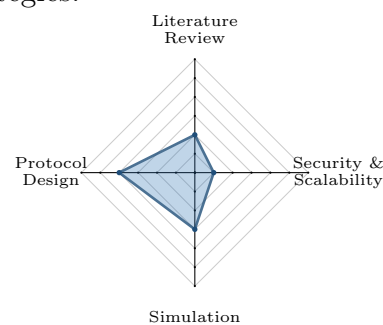
- Literature Review: 50%
- Protocol Design: 20%
- Simulation: 20%
- Security & Scalability: 10%

**W.D.A.C. Bandara - 230082G****Main Area:** Protocol Design

- **Literature Review:** Identified scalability and metric-related limitations in existing protocols. Contributed to comparative evaluation of routing techniques.
- **Protocol Design:** Played a major role in designing the AOSPF architecture. Developed adaptive composite routing metric formulation. Designed threshold-based triggering mechanisms. Contributed to packet format design, routing logic, and pseudocode development. Led discussions on protocol workflow and system behavior.
- **Simulation:** Assisted in validating simulation behavior against design specifications. Helped analyze routing performance outputs.
- **Security & Scalability:** Contributed ideas for improving routing stability and resilience. Assisted in reviewing robustness strategies.

Contribution Breakdown:

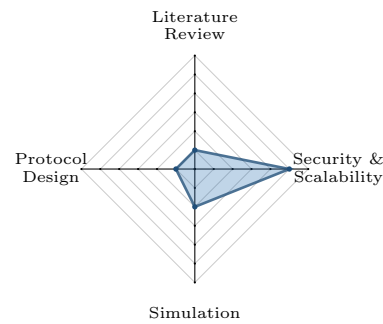
- Literature Review: 20%
- Protocol Design: 40%
- Simulation: 30%
- Security & Scalability: 10%

**R.K.T. Dissanayake - 230164K****Main Area:** Security & Scalability Analysis

- **Literature Review:** Analyzed security-related weaknesses of RIP, OSPF, IS-IS, and BGP. Contributed to discussions on routing attacks, spoofing, and scalability concerns.
- **Protocol Design:** Contributed ideas related to adaptive routing stability and secure routing mechanisms. Assisted in discussions on threshold-based update triggering and protocol robustness.
- **Simulation:** Assisted in validating simulation outputs against protocol specifications. Helped analyze routing performance and convergence behavior.
- **Security & Scalability:** Led discussions on scalability optimization and routing stability. Developed and documented the HMAC-based authentication mechanism. Contributed to replay attack prevention and spoofing protection. Evaluated protocol robustness under failure and attack scenarios. Analyzed scalability limitations and proposed improvement strategies.

Contribution Breakdown:

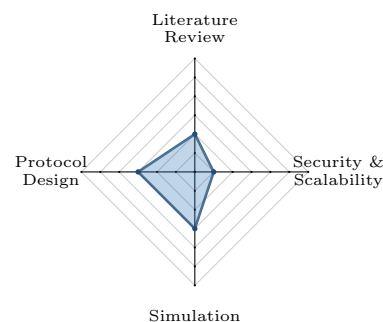
- Literature Review: 15%
- Protocol Design: 15%
- Simulation: 20%
- Security & Scalability: 50%

**U.G.R.B. Tennakoon - 230629R****Main Area:** Simulation

- **Literature Review:** Assisted in collecting technical references on routing protocols. Contributed to reviewing comparative analyses and validating technical descriptions.
- **Protocol Design:** Provided implementation-oriented feedback during protocol design. Assisted in evaluating feasibility of proposed routing mechanisms.
- **Simulation:** Implemented the AOSPF simulation using Python and NetworkX. Developed routing logic, cost calculations, and threshold-based triggering mechanisms. Handled event processing and protocol behavior modeling. Performed debugging and refinement. Evaluated convergence behavior and routing performance metrics.
- **Security & Scalability:** Assisted in testing failure scenarios. Helped evaluate robustness under adversarial and unstable network conditions.

Contribution Breakdown:

- Literature Review: 20%
- Protocol Design: 35%
- Simulation: 35%
- Security & Scalability: 10%

**Overall Team Collaboration**

All members actively contributed across all phases of the project. The team engaged in continuous technical discussions, design refinement, simulation debugging, result analysis, and documentation preparation. The final system was achieved through collaborative decision-making and iterative improvement across all components.

References

- [1] J. Moy, “OSPF Version 2,” RFC 2328, Apr. 1998. <https://datatracker.ietf.org/doc/html/rfc2328>
- [2] R. Coltun, D. Ferguson, J. Moy, and A. Lindem, “OSPF for IPv6,” RFC 5340, Jul. 2008. <https://datatracker.ietf.org/doc/html/rfc5340>
- [3] G. Malkin, “RIP Version 2,” RFC 2453, Nov. 1998. <https://datatracker.ietf.org/doc/html/rfc2453>
- [4] S. Murphy, “BGP Security Vulnerabilities Analysis,” RFC 4272, Jan. 2006. <https://datatracker.ietf.org/doc/html/rfc4272>
- [5] Y. Rekhter, T. Li, and S. Hares, “A Border Gateway Protocol 4 (BGP-4),” RFC 4271, Jan. 2006. <https://datatracker.ietf.org/doc/html/rfc4271>
- [6] C. Perkins, E. Belding-Royer, and S. Das, “Ad hoc On-Demand Distance Vector (AODV) Routing,” RFC 3561, Jul. 2003. <https://datatracker.ietf.org/doc/html/rfc3561>
- [7] S. Kent and K. Seo, “Security Architecture for the Internet Protocol,” RFC 4301, Dec. 2005. <https://datatracker.ietf.org/doc/html/rfc4301>

A. Supplementary Materials

The simulation implementation is available at:

<https://github.com/Achintha-Bandara/Routing-Protocol-Design.git>